

Documentación Técnica

Microservicio: ROLE-MANAGE

Sistema de Gestión de Co-working

Ingeniería de Computación | 7mo Semestre

Integrantes del equipo	Información de entrega
1. <u>Samuel Rojas</u>	Lenguaje / Framework:
2. <u>Brian Berbey</u>	Puerto del servicio: <u>:8000</u>
3. <u>Leonardo correa</u>	Fecha de entrega: <u>02/06/2026</u>
4. _____	Link del repositorio:
5. _____	<u>https://github.com/lpzzlnrd/Coworking.git</u>

1 Descripción del microservicio

1.1 Responsabilidad del servicio

Describan en 3-5 oraciones que hace este servicio y que NO hace (sus límites).

Gestiona identidad y control de acceso: registro/login, hashing de contraseñas, emisión y verificación de JWT, y endpoints de autenticación y usuarios.
Persiste usuarios y roles en Postgres y expone dependencias reutilizables para proteger rutas (<u>get_current_user</u> , <u>require_role</u>).
Proporciona operaciones administrativas básicas sobre usuarios: listar, obtener, y actualizar rol/estado.
No integra SSO ni proveedores externos de identidad, ni gestiona envío de correos (verificación/recuperación).
No implementa autorización fina por recurso ni sesiones server-side, y no es responsable de la interfaz de usuario.

1.2 ¿Por qué existe como microservicio separado?

Justifiquen la separación: ¿qué problema tendríamos si esta lógica viviera en otro servicio?

Aislamiento de responsabilidades: centraliza la lógica sensible de identidad (hashing, emisión/validación de JWT, gestión de roles) y así reduce la superficie de ataque.
Independencia de despliegue y escalado: permite escalar o desplegar el servicio de autenticación por separado según carga sin afectar otros servicios.
Ciclo de vida y seguridad independiente: facilita rotación de secretos, actualizaciones de seguridad y políticas de acceso sin tocar dominios de negocio.
Reutilización y consistencia: un único servicio garantiza tokens y reglas de autorización uniformes para frontend y microservicios consumidores.
Evita acoplamiento y duplicación: mover esta lógica a otro servicio introduciría dependencias cruzadas y duplicación de código, complicando mantenimiento y auditoría.

1.3 Integración con otros servicios

Con quien se comunica, como verifica el JWT y que datos intercambia.

Se comunica principalmente con el frontend y otros consumidores del API a través de HTTP en los endpoints /auth/ y /users/.
Verifica el JWT en (app/deps.py) usando decode_access_token() de (app/security/jwt.py), comprobando firma, expiración y el campo sub.
El JWT lleva "sub", "role", "iat" y "exp", y el servicio devuelve "access_token" y "expires_in" en el login.
Internamente también intercambia datos de usuarios con la base de datos Postgres (User), consultando email, rol y estado activo.
Otros servicios usan ese token para autenticar peticiones y decidir permisos, siempre que compartan la misma clave/secreto para validar JWT.

2 Tabla de endpoints

2.1 Endpoints del servicio

Metodo	URL	Descripcion	Token	Rol
GET	http://<ROLE-MANAGE-HOST>:8000/health	Estado del servicio	NO	n/a

POST	http://<ROLE-MANAGE-HOST>:8000/auth/register	Registra un nuevo usuario	NO	n/a
POST	http://<ROLE-MANAGE-HOST>:8000/auth/login	Login y emisión de JWT	NO	n/a
GET	http://<ROLE-MANAGE-HOST>:8000/auth/me	Devuelve el usuario autenticado	SI	cualquiera
GET	http://<ROLE-MANAGE-HOST>:8000/auth/verify	Verifica JWT y muestra claims básicos	SI	cualquiera
GET	http://<ROLE-MANAGE-HOST>:8000/users	Lista usuarios paginados	SI	solo admin
GET	http://<ROLE-MANAGE-HOST>:8000/users/{user_id}	Obtiene un usuario por ID	SI	admin o staff
PATCH	http://<ROLE-MANAGE-HOST>:8000/users/{user_id}/role	Actualiza el rol de un usuario	SI	solo admin

2.2 Ejemplo completo — request y response del endpoint principal

Elijan el endpoint más importante y muestren el JSON completo.

Método y URL del endpoint elegido

```
// Request body:
POST /auth/register
{
  "email": "user@example.com",
  "password": "supersecret123",
  "full_name": "Usuario Ejemplo"
}
POST /auth/login
{
  "email": "user@example.com",
  "password": "supersecret123"
}
GET /auth/me

PATCH /users/{user_id}/role
{
  "role": "admin"
```

```
}
```

```
// Response:
POST /auth/register
{
  "id": "a2f90d88-...-b5c7",
  "email": "user@example.com",
  "full_name": "Usuario Ejemplo",
  "role": "member",
  "is_active": true,
  "created_at": "2026-06-03T12:34:56.789000"
}
POST /auth/login
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer",
  "expires_in": 900
}
GET /auth/me
{
  "id": "a2f90d88-...-b5c7",
  "email": "user@example.com",
  "full_name": "Usuario Ejemplo",
  "role": "member",
  "is_active": true,
  "created_at": "2026-06-03T12:34:56.789000"
}
PATCH /users/{user_id}/role
{
  "id": "a2f90d88-...-b5c7",
  "email": "user@example.com",
  "full_name": null,
  "role": "admin",
  "is_active": true,
  "created_at": "2026-06-03T12:34:56.789000"
}
```

3 Análisis del algoritmo implementado

3.1 Nombre y descripción del algoritmo

Nombre, en que archivo vive y que problema resuelve en el sistema.

Nombre: Autenticación y autorización de usuario.
Archivo: auth.py

Qué hace: implementa el flujo de login y validación de sesión; busca el usuario por email, verifica la contraseña con Argon2, genera un JWT firmado, y luego valida ese token en endpoints protegidos.

Problema que resuelve: asegura que solo usuarios válidos puedan acceder al sistema y que cada petición porte credenciales verificadas, evitando acceso no autorizado y permitiendo control de roles para rutas administrativas.

3.2 Como funciona paso a paso

Explicación en lenguaje natural. Pueden usar diagrama, pseudocodigo o pasos numerados.

En /auth/login, el servicio normaliza el email a minúsculas y ejecuta una consulta SQL `SELECT User WHERE email = ...` usando el índice único en `users.email`.

Si no encuentra usuario o la contraseña no coincide con `verify_password` (Argon2), devuelve 401; si el usuario existe pero `is_active` es falso, devuelve 403.

Si es válido, crea un JWT con `sub = user.id`, `role` y `exp`, lo firma con `JWT_SECRET` y lo devuelve al cliente.

En cada endpoint protegido, `get_current_user` decodifica el JWT con `decode_access_token`, valida firma y expiración, extrae `sub`, lo convierte a UUID y busca el usuario por PK en la base de datos.

Luego `require_role` compara el rol actual contra los roles permitidos y deja pasar solo si coincide.

3.3 Comparación de complejidad O()

Llenen la tabla comparativa con su algoritmo y la alternativa sin algoritmo.

Su algoritmo	Alternativa naive (el for simple)
Nombre: Consulta indexada + verificación JWT	Nombre: Búsqueda secuencial de usuario en memoria
Complejidad: $O(\log n)$	Complejidad: $O(n)$

Con 100 elementos hace: Busca en el índice en $\sim \log_2(100)$ pasos, recupera 1 fila y verifica contraseña/token en $O(1)$	Con 100 elementos hace: Revisa hasta 100 registros, comparando email y potencialmente verificando contraseña en cada uno
Con 10,000 elementos hace: Busca en el índice en $\sim \log_2(10000)$ pasos, recupera 1 fila y verifica contraseña/token en $O(1)$	Con 10,000 elementos hace: Revisa hasta 10,000 registros, comparando email y potencialmente verificando contraseña en cada uno

3.4 Ejemplo numérico con datos reales del sistema

Usen un ejemplo concreto de su servicio: reservas, espacios, facturas, etc.

En el servicio role-manage tenemos una tabla users con 10,000 usuarios registrados.
Cuando un cliente hace POST /auth/login con email = "user10@example.com" y contraseña correcta, la consulta indexada busca ese email entre 10,000 filas y devuelve directamente el registro del usuario.
Luego se verifica la contraseña con Argon2 en tiempo constante y se genera un JWT; en total el costo real es de unas pocas decenas de operaciones, no de miles.
Si en cambio usáramos el enfoque naive, tendríamos que recorrer todos los usuarios hasta encontrar el email, es decir hasta 10,000 comparaciones de strings y quizá varias verificaciones de contraseña fallida.
Para un caso más pequeño, con 100 usuarios el algoritmo indexado hace ~ 7 pasos de búsqueda en índice ($\log_2(100)$) mientras que la alternativa naive podría revisar hasta 100 registros; con 10,000 usuarios la diferencia es ~ 14 pasos frente a 10,000 comparaciones.

4 Arquitectura y decisiones técnicas

4.1 Diagrama de arquitectura del servicio

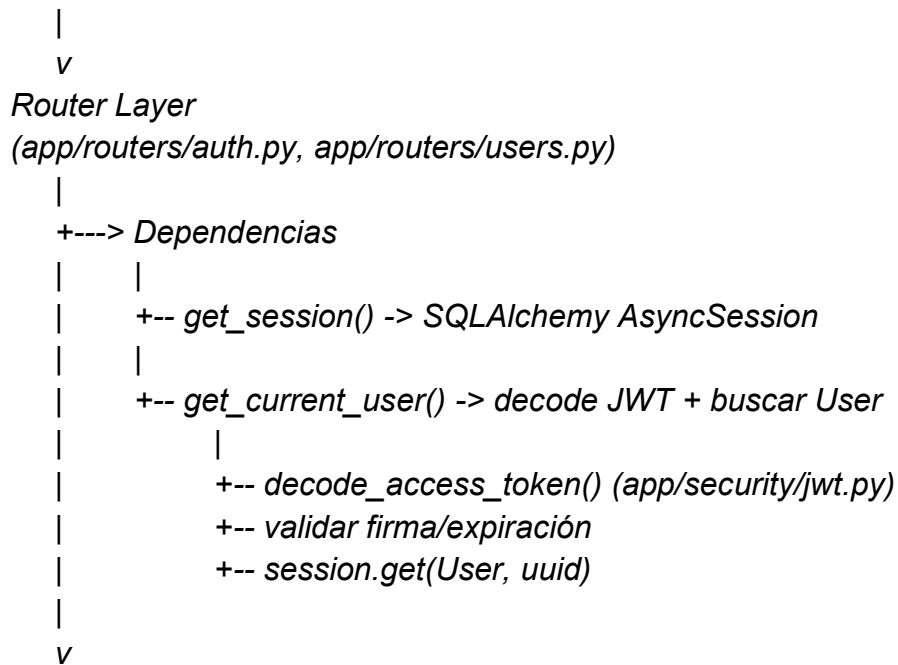
Muestren las capas internas: como fluye una request desde que entra hasta que devuelve respuesta.

Cliente HTTP

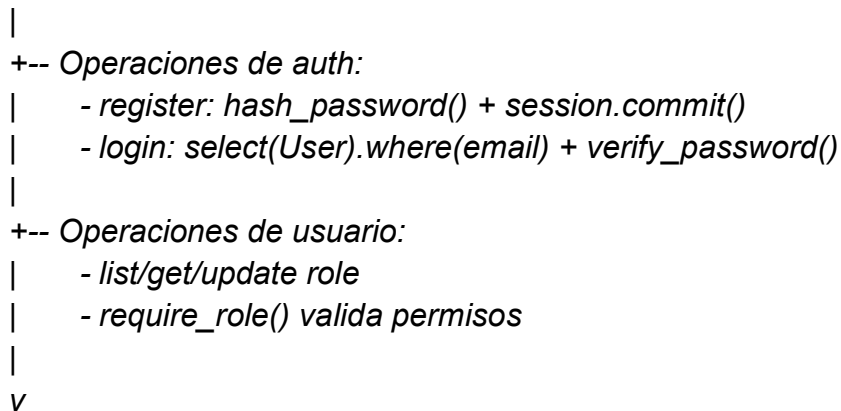
|
v

Uvicorn / FastAPI app

(app/main.py create_app)



Lógica de negocio / validación
 (app/routers/... + app/schemas/...)



Persistencia
 (app/db.py + Postgres)

Respuesta Pydantic
 (app/schemas/*.py)

FastAPI retorna JSON al cliente

4.2 Por que eligieron este lenguaje para este servicio

No es suficiente decir que lo conocen. Justifiquen porque es apropiado para la responsabilidad de este servicio.

Python es apropiado porque el servicio es una API de backend pequeña y con lógica de autenticación, donde se necesita velocidad de desarrollo y legibilidad más que un sistema de alto rendimiento extremo.

FastAPI facilita la construcción de endpoints, validación de request/response con Pydantic (app/schemas/*.py), y la inyección de dependencias para seguridad (app/deps.py), lo que encaja con este servicio de control de acceso.

El servicio usa I/O intensivo con base de datos asíncrona (asyncpg, SQLAlchemy asyncio en app/db.py), y Python/ FastAPI soporta async de forma nativa, reduciendo bloqueo en consultas y permitiendo manejar concurrencia de login/verify sin complicar la arquitectura.

La comunidad de Python ofrece las librerías de seguridad necesarias de forma madura: JWT con pyjwt, hashing seguro con Argon2, y manejo de errores DB, lo que es ideal para un servicio que protege credenciales.

4.3 Como funciona la verificación JWT en este servicio

Paso a paso: como llega el token, como se verifica y que datos se extraen del payload.

El token llega en el header Authorization: Bearer <token> cuando el cliente llama un endpoint protegido.

FastAPI inyecta Depends(get_current_user) en el endpoint; get_current_user recibe el token desde oauth2_scheme.

get_current_user llama decode_access_token(token) en app/security/jwt.py, que usa jwt.decode(..., settings.JWT_SECRET, algorithms=[settings.JWT_ALGORITHM]).

Si la firma o expiración fallan, decode_access_token lanza TokenError y get_current_user devuelve 401.

Si el token es válido, get_current_user extrae sub del payload, lo convierte a UUID y carga el usuario desde la base de datos.

Luego verifica que el usuario exista y esté activo; si no, también falla. Los datos que se extraen del payload son sub (ID del usuario), role, iat y exp; aquí sub se usa para buscar el usuario, y role puede usarse luego para autorización con require_role().

4.4 Una decisión técnica importante que tomaron

Algo que decidieron hacer de cierta manera y podría haberlo hecho diferente. Expliquen el tradeoff.

técnica importante: usar JWT firmado localmente con HS256 y validar el token dentro del mismo servicio en `app/security/jwt.py`, en lugar de delegar cada verificación a un servicio de auth central.

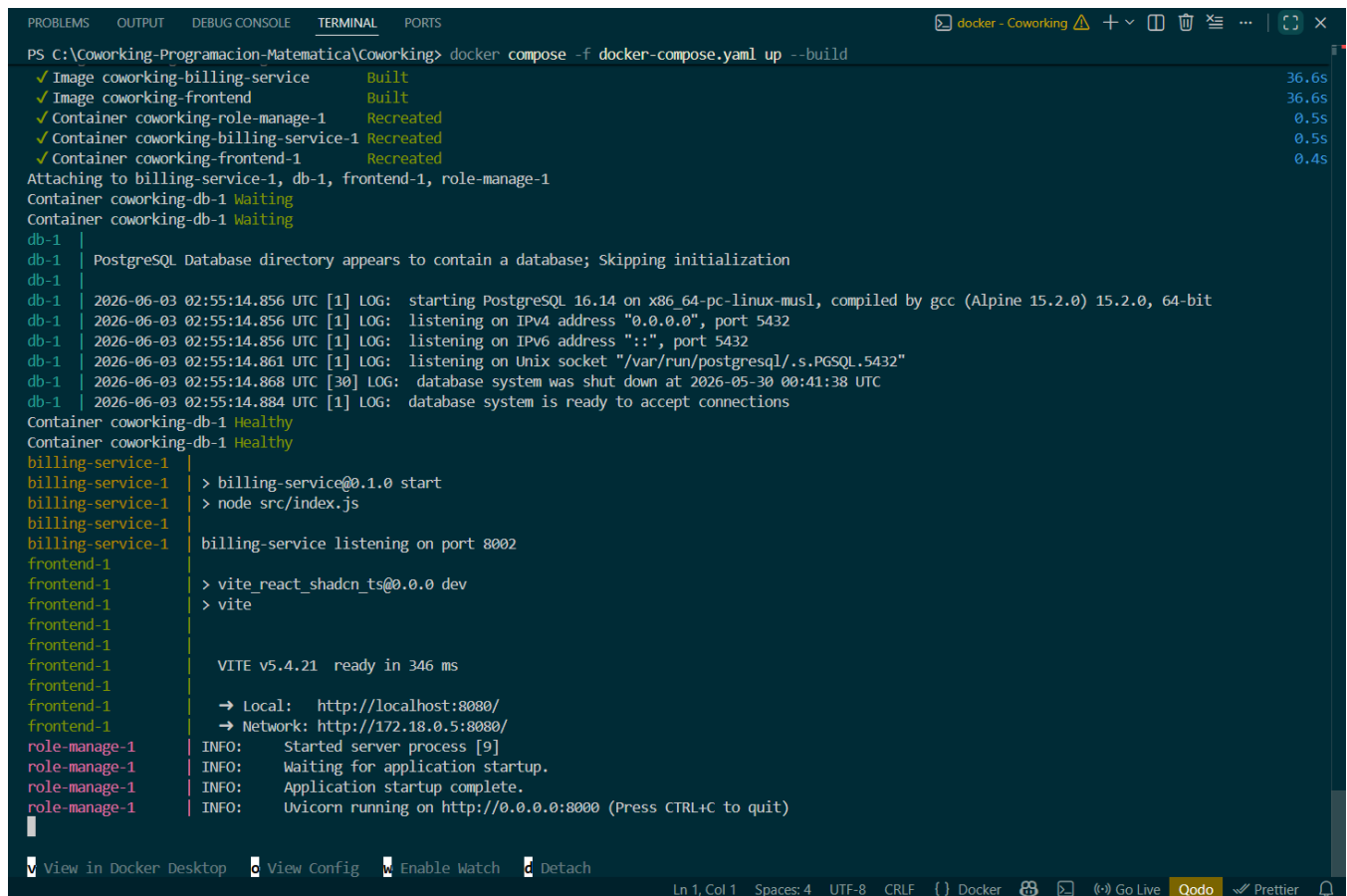
Por qué: esto reduce la latencia y la dependencia de red en cada request protegida, permitiendo que `get_current_user()` valide token en $O(1)$ y luego sólo haga una consulta por sub en la DB.

Tradeoff: a favor está la simplicidad y el rendimiento, pero en contra está que la revocación inmediata de tokens es más difícil; si el secreto se compromete o se quiere invalidar un token antes de expiración, se necesita un mecanismo extra (lista negra, cambio de clave global).

Alternativa: usar un auth service dedicado para validar sesiones permitiría revocación centralizada y auditoría, pero aumentaría la complejidad y el costo de cada request protegido.

5 Evidencia de funcionamiento

Captura 1 — Docker levantando el servicio



```
PS C:\Coworking-Programacion-Matematica\Coworking> docker compose -f docker-compose.yaml up --build
[+] Image coworking-billing-service Built 36.6s
[+] Image coworking-frontent Built 36.6s
[+] Container coworking-role-manage-1 Recreated 0.5s
[+] Container coworking-billing-service-1 Recreated 0.5s
[+] Container coworking-frontent-1 Recreated 0.4s
Attaching to billing-service-1, db-1, frontend-1, role-manage-1
Container coworking-db-1 Waiting
Container coworking-db-1 Waiting
db-1 |
db-1 | PostgreSQL Database directory appears to contain a database; Skipping initialization
db-1 |
db-1 | 2026-06-03 02:55:14.856 UTC [1] LOG: starting PostgreSQL 16.14 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit
db-1 | 2026-06-03 02:55:14.856 UTC [1] LOG: listening on IPv4 address "0.0.0.0", port 5432
db-1 | 2026-06-03 02:55:14.856 UTC [1] LOG: listening on IPv6 address ":::", port 5432
db-1 | 2026-06-03 02:55:14.861 UTC [1] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
db-1 | 2026-06-03 02:55:14.868 UTC [30] LOG: database system was shut down at 2026-05-30 00:41:38 UTC
db-1 | 2026-06-03 02:55:14.884 UTC [1] LOG: database system is ready to accept connections
Container coworking-db-1 Healthy
Container coworking-db-1 Healthy
billing-service-1 |
billing-service-1 | > billing-service@0.1.0 start
billing-service-1 | > node src/index.js
billing-service-1 |
billing-service-1 | billing-service listening on port 8002
frontend-1 |
frontend-1 | > vite_react_shadcn_ts@0.0.0 dev
frontend-1 | > vite
frontend-1 |
frontend-1 | VITE v5.4.21 ready in 346 ms
frontend-1 |
frontend-1 | → Local: http://localhost:8080/
frontend-1 | → Network: http://172.18.0.5:8080/
role-manage-1 | INFO: Started server process [9]
role-manage-1 | INFO: Waiting for application startup.
role-manage-1 | INFO: Application startup complete.
role-manage-1 | INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

Captura 2 — Endpoint funcionando con respuesta JSON

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
● PS C:\Coworking-Programacion-Matematica\Coworking> $email = "captura-role-" + (Get-Date -Format "yyyyMMddHHmmss") + "@example.com"
>>
>> $body = @{
>>     email = $email
>>     password = "hunter22!"
>>     full_name = "Usuario Captura Role"
>> } | ConvertTo-Json
>>
>> Invoke-RestMethod -Method POST `
>>     -Uri "http://localhost:8001/auth/register" `
>>     -ContentType "application/json" `
>>     -Body $body

id           : 630e9777-0007-4f42-b38c-764bdf5c96b7
email        : captura-role-20260602234722@example.com
full_name    : Usuario Captura Role
role         : member
is_active    : True
created_at   : 2026-06-03T03:47:22.374615Z

○ PS C:\Coworking-Programacion-Matematica\Coworking> █
```

Captura 3 — Health check del servicio

```
● PS C:\Coworking-Programacion-Matematica\Coworking> Invoke-RestMethod http://localhost:8001/health

status service  env
-----
ok      role-manage dev

○ PS C:\Coworking-Programacion-Matematica\Coworking> █
```

6 Reflexión final del equipo

Lo más difícil fue integrar correctamente la verificación JWT con las dependencias de FastAPI y la sesión asíncrona de SQLAlchemy; el flujo Depends(get_current_user) + AsyncSession tiene varias partes y es fácil equivocarse en el orden de las llamadas o en la gestión de errores. Aprendimos que FastAPI hace muy cómodo el manejo de esquemas y autenticación, pero que hay que prestar atención a cómo se inyectan dependencias asíncronas y a cómo Pydantic serializa los modelos SQLAlchemy; también quedó claro que PYDANTIC_SETTINGS simplifica la configuración de env vars. Con más tiempo mejoraríamos la seguridad y la robustez: agregar refresh tokens, manejo de revocación de JWT, endpoints de recuperación de contraseña, logging de auditoría, pruebas de integración con Alembic y un endpoint de health que chequee la base de datos, no solo el servicio.
